Rapid time series prediction with a hardware-based reservoir computer

Daniel Canaday, Aaron Griffith, and Daniel J. Gauthier
Department of Physics, Ohio State University, 191 West Woodruff Ave., Columbus, Ohio 43210, USA

(Received 11 July 2018; accepted 20 November 2018; published online 20 December 2018)

Reservoir computing is a neural network approach for processing time-dependent signals that has seen rapid development in recent years. Physical implementations of the technique using optical reservoirs have demonstrated remarkable accuracy and processing speed at benchmark tasks. However, these approaches require an electronic output layer to maintain high performance, which limits their use in tasks such as time-series prediction, where the output is fed back into the reservoir. We present here a reservoir computing scheme that has rapid processing speed both by the reservoir and the output layer. The reservoir is realized by an autonomous, time-delay, Boolean network configured on a field-programmable gate array. We investigate the dynamical properties of the network and observe the fading memory property that is critical for successful reservoir computing. We demonstrate the utility of the technique by training a reservoir to learn the short- and long-term behavior of a chaotic system. We find accuracy comparable to state-of-the-art software approaches of a similar network size, but with a superior real-time prediction rate up to 160 MHz. *Published by AIP Publishing.* https://doi.org/10.1063/1.5048199

Reservoir computers are well-suited for machine learning tasks that involve processing time-varying signals such as those generated by human speech, communication systems, chaotic systems, weather systems, and autonomous vehicles. Compared to other neural network techniques, reservoir computers can be trained using less data and in much less time. They also possess a large network component, called the reservoir, that can be re-used for different tasks. These advantages have motivated searches for physical implementations of reservoir computers that achieve high-speed and real-time information processing, including opto-electronic and electronic devices. Here, we develop an electronic approach using an autonomous, time-delay, Boolean network configured on a field-programmable gate array (FPGA). These devices allow for complex networks consisting of 1000s of nodes with an arbitrary network topology. Time-delays can be incorporated along network links, thereby allowing for extremely high-dimension reservoirs. The characteristic time scale of a network node is less than a nanosecond, allowing for information processing in the GHz regime. Furthermore, because the reservoir state is Boolean rather than real-valued, the calculation of an output from the reservoir state can be done rapidly with synchronous FPGA logic. We use such a reservoir computer for the challenging task of forecasting the dynamics of a chaotic system. This work paves the way for low-cost, compact reservoir computers that can be embedded in various commercial and industrial systems for real-time information processing.

I. INTRODUCTION

There is considerable interest in the machine learning community in using recurrent neural networks (RNNs) for processing time-dependent signals.[1–3] Under some mild assumptions, these types of networks are universal approximators of dynamical systems,[4] similarly to how multilayer feedforward neural networks are universal approximators of static maps.[5] Many machine learning and artificial intelligence tasks, such as dynamical system modeling, human speech recognition, and natural language processing are intrinsically time-dependent tasks, and thus are more naturally handled within a time-dependent, neural-network framework.

Though they have high expressive power, RNNs are difficult to train using gradient-descent-based methods.[6] One approach to efficiently and rapidly train an RNN is known as reservoir computing (RC). In RC, the network is divided into input nodes, a bulk collection of nodes known as the *reservoir*, and output nodes, such that the only recurrent links are between reservoir nodes. Training involves only adjusting the weights along links connecting the reservoir to the output nodes and not the recurrent links in the reservoir. This approach displays state-of-the-art performance in a variety of time-dependent tasks, including chaotic time series prediction,[7] system identification and control,[8] and spoken word recognition,[9] all with remarkably short training times in comparison to other neural-network approaches.

Recently, implementations of reservoir computing using dedicated hardware have achieved much attention, particularly those based on delay-coupled photonic systems.[10–12] These devices allow for reservoir computing at extremely high speeds, including the classification of spoken words at a rate of millions of words per second.[13] There is also the potential to form the input and output layers out of optics as well, resulting in an all-optical computational device.[14,15] However, these devices are not well-equipped to handle tasks such as time-series prediction, which require the input and output layers to be coupled.

Here, we present a hardware implementation of RC based on an autonomous, time-delay, Boolean network realized on a readily-available platform known as a field-programmable

gate array (FPGA). This approach allows for a seamless coupling of reservoir to the output due to the spatially simple nature of the reservoir state and the fact that matrix multiplication can be realized with compact Boolean logic. Together with the parallel nature of the network, this allows for up to 10 times faster information processing than delay-coupled photonic devices.[13] We apply our implementation to the challenging task of predicting the behavior of a chaotic dynamical system. We find prediction accuracy similar to software-based techniques of similar network size and achieve a record-high real-time prediction rate.[16]

The rest of this article is organized as follows: we describe the RC technique in general terms, detailing the necessary components and their features in Sec. II; we discuss our approach to realizing these features in an efficient manner on an FPGA in Secs. III–V; we discuss the performance of our approach for prediction of the Mackey-Glass system in Secs. VI–VII; and we conclude with a discussion of our results in Sec. VIII.

II. RESERVOIR COMPUTING FOR TIME-SERIES PREDICTION

Reservoir computing is a concept introduced independently by Jaeger[17] and Maass[18] under the names Echo State Network (ESN) and Liquid State Machine (LSM), respectively. In Jaeger's technique, a network of recurrently connected sigmoidal nodes (the reservoir) with state $\mathbf{X}(t)$ is excited by a time-dependent input signal $\mathbf{u}(t)$. The reservoir is observed during some training period, an approximate linear transformation from $\mathbf{X}(t)$ to a desired signal $\mathbf{v}_d(t)$ is identified via linear regression, and this linear transformation forms the readout layer. These signals and their relations to one another are illustrated in Fig. 1(a). The LSM technique has the same features, but uses a pool of spiking nodes to form the reservoir.

The two approaches described by Jaeger and Maass are apparently similar, and indeed are two particular implementations of RC. As a class of techniques, RC can be defined quite broadly, and we do so as follows. Given an input signal $\mathbf{u}(t)$ and a desired output signal $\mathbf{v}_d(t)$, a reservoir computer constructs a mapping from $\mathbf{u}(t)$ to $\mathbf{v}_d(t)$ with the following steps:

- create a randomly parameterized network of nodes and recurrent links called the *reservoir* with state $\mathbf{X}(t)$ and dynamics described by $\dot{\mathbf{X}}(t) = \mathbf{f}\left[\mathbf{X}(t), \mathbf{u}(t)\right]$;
- excite the reservoir with an input signal $\mathbf{u}(t)$ over some training period and observe the response of the reservoir;
- form a readout layer that transforms the reservoir state $\mathbf{X}(t)$ to an output $\mathbf{v}(t)$, such that $\mathbf{v}(t)$ well approximates $\mathbf{v}_d(t)$ during the training period.

Figure 1(a) contains a schematic representation of the resulting system, which consists of the reservoir, the input signal, the trained readout layer, and output signal. Note that we make no assumptions about the dynamics $\mathbf{f}$. In general, it may include discontinuities, time-delays, or have components simply equal to $\mathbf{u}(t)$ (i.e., the reservoir may include a direct connection from the input to the output).

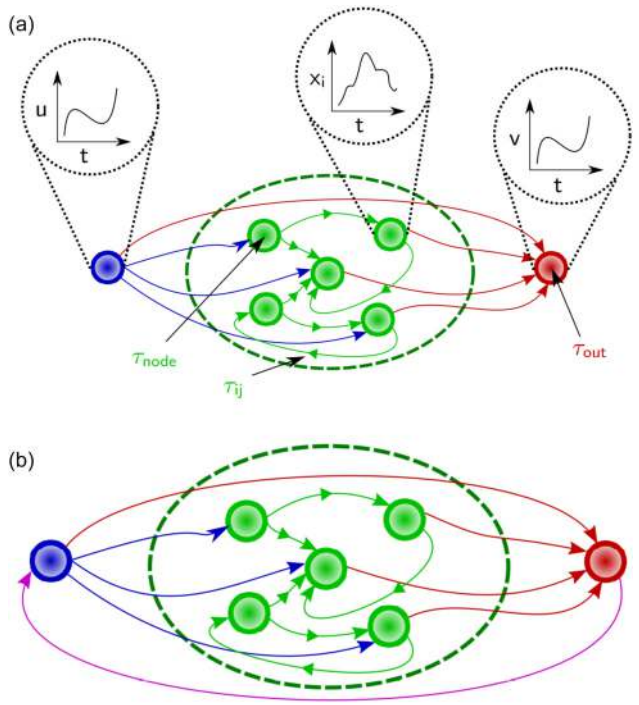

FIG. 1. Schematic representation of the RC scheme. (a) A general reservoir computer learns to map an input onto a desired output. The network dynamics may contain propagation delays along the links (denoted by τ_{ij}) or through nodes (such as through the output layer, denoted by τ_{out}). (b) For the particular task of predicting a signal, the reservoir is trained so that the target output is equal to the input. After training, the output is fed back into the reservoir, resulting in an autonomous dynamical system. If properly trained, the autonomous reservoir serves as a model for the dynamics that generated the input signal.

Reservoir computing demonstrates remarkable success at predicting a chaotic time series, among other applications. The goal of this task is to predict the output of an unknown dynamical system after a training period. In the context of RC, this is accomplished by setting $\mathbf{v}_d(t) = \mathbf{u}(t)$, i.e., by training the reservoir computer to reproduce its inputs. Then, after training is complete, we replace $\mathbf{u}(t)$ with $\mathbf{v}(t)$ and allow the newly-formed autonomous system to evolve in time beyond the end of the training period. This closed-loop system is illustrated in Fig. 1(b) and consists of the same components as in the general picture, but the input and output are the same signal. This scheme can predict accurately the short-term behavior of a variety of systems, including the Mackey-Glass,[17] Lorenz,[19] and Kuramoto-Sivashinsky spatial-temporal[19] systems using a software simulation of the reservoir. A reservoir computer trained in this manner can also learn the long-term behavior of complex systems, generating the true attractor of the target system and replicating its Lyapunov spectrum.[19]

Although training the network consists only of identifying optimal parameters in the readout layer, there are a variety of factors in designing the reservoir that impact the success of the scheme. These factors include:

Matching time scales. In general, both the reservoir and the source of the signal $\mathbf{u}(t)$ are dynamical systems with their own characteristic time scales. These time scales must be similar for the reservoir to produce $\mathbf{v}(t)$.[20] For software based

approaches to RC, these scales are matched by tuning the reservoir's temporal properties through accessible reservoir parameters, such as the response time τ_{node} of reservoir nodes. However, with hardware-based approaches, the parameters controlling the time-scale of reservoir dynamics are often more rigid. We compensate for this by adjusting the time scale of the input signal (see Sec. IV) and adding delays to the links within the reservoir (see Sec. III A).

Reservoir memory. It is generally believed, as was observed by Jaeger and Maass in their respective architectures and as has been explored more generally,[21] that a good reservoir for RC is a system that possesses *fading memory*. That is, the reservoir state contains information about the input signal $\mathbf{u}(t)$, but the effect of small differences in $\mathbf{u}(t)$ dissipates over time. This is often referred to as the echo-state property in the context of ESNs and is described in greater detail in Sec. III B. We find that the autonomous reservoirs considered here have the fading memory property. Furthermore, we find that the characteristic time scale over which small differences dissipate can be tuned by adding delays to the links within the reservoir.

Coupling to input. Each RC implementation couples $\mathbf{u}(t)$ to the reservoir in a very technique-dependent way, such as spike-encoding in LSMs or by consideration of so-called "virtual nodes" in photonic reservoirs.[22] The coupling in our FPGA-based approach is complicated by the fact that nodes execute Boolean functions, whereas the input signal $\mathbf{u}(t)$ is a large-bit representation of a real number. We must also consider, as with most techniques for processing physical data, the limited precision and sampling rate of the input signal. The sampling rate is particularly relevant for our physical reservoir computer, as the reservoir nodes have their own, fixed characteristic time scale. These issues are discussed in Sec. IV.

Calculating $\mathbf{v}(t)$. In software-based reservoir computing schemes, the readout layer performs its operation effectively instantaneously as far as the simulation is concerned. However, this is not possible when the reservoir is a continuously-evolving physical system. There is a finite time required to calculate $\mathbf{v}(t)$, which can be interpreted as a propagation delay τ_{out} [see Fig. 1(a)] through the readout layer and ultimately limits the rate at which predictions can be made in closed-loop operation. Consequently, $\mathbf{v}(t)$ must be calculated from a measurement of $\mathbf{X}(t-\tau_{out})$ for the predicted output to be ready to be fed back into the input at time t.

The goal of this work is to demonstrate a technique for realizing a hardware implementation of RC with a minimal output delay so that predictions can be made as rapidly as possible. In Secs. III–VII, we detail the construction of the various components of the reservoir computer illustrated in Fig. 1 and how they address the general RC properties outlined in Secs. III–V.

III. AUTONOMOUS BOOLEAN RESERVOIR

We propose a reservoir construction based on an autonomous, time-delay, Boolean reservoir realized on an FPGA. By forming the nodes of the reservoir out of FPGA elements themselves, this approach exhibits faster computation than FPGA-accelerated neural networks,[23,24] which require explicit multiplication, addition, and non-linear transformation calculations at each time-step. Our approach also has the advantage of realizing the reservoir and the readout layer on the same platform without delays associated with transferring data between different hardware. Finally, due to the Boolean-valued state of the reservoir, a linear readout layer $\left[\mathbf{v}(t) = \mathbf{W}_{out}\mathbf{X}(t)\right]$ is reduced to an addition of real numbers rather than a full matrix multiplication. This allows for much shorter total calculation time and thus faster real-time prediction than in opto-electronic RC.[16]

Our choice of reservoir is further motivated by the observation that Boolean networks with time-delay can exhibit complex dynamics, including chaos.[25] In fact, a single XOR node with delayed feedback can exhibit a fading memory condition and is suitable for RC on simple-tasks such as binary pattern recognition.[26]

It has been proposed that individual FPGA nodes have dynamics that can be described by the Glass model[27] given by

$$\gamma_i \dot{x}_i = -x_i + \Lambda_i(X_{i1}, X_{i2}, \ldots), \tag{1}$$

$$X_i = \begin{cases} 1 & \text{if } x_i \geq q_i, \\ 0 & \text{if } x_i < q_i, \end{cases} \tag{2}$$

where x_i is the continuous variable describing the state of the node, γ_i describes the time-scale of the node, q_i is a thresholding variable, and Λ_i is the Boolean function assigned to the node. The thresholded Boolean variable X_{ij} is the j_{th} input to the i_{th} node.

We construct our Boolean reservoir by forming networks of nodes described by Eqs. (1) and (2) and the Boolean function

$$\Lambda_i = \Theta\left(\sum_j W^{ij}X_j + W^{ij}_{in}u_j\right), \tag{3}$$

where u_j are the bits of the input vector $\mathbf{u}$, $\mathbf{W}$ is the reservoir-reservoir connection matrix, $\mathbf{W}_{in}$ is the input-reservoir connection matrix, and Θ is the Heaviside step function defined by

$$\Theta(x) = \begin{cases} 1 & \text{if } x > 0, \\ 0 & \text{if } x \leq 0. \end{cases} \tag{4}$$

The matrices $\mathbf{W}$ and $\mathbf{W}_{in}$ are chosen as follows. Each node receives the input from exactly k other randomly chosen nodes, thus determining k non-zero elements of each row of $\mathbf{W}$. The non-zero elements of $\mathbf{W}$ are given a random value from a uniform distribution between -1 and 1. The maximum absolute eigenvalue (spectral radius) of the matrix $\mathbf{W}$ is calculated and used to scale $\mathbf{W}$ such that its spectral radius is ρ. A proportion σ of the nodes are chosen to receive input, thus determining the number of non-zero rows of $\mathbf{W}_{in}$. The non-zero values of $\mathbf{W}_{in}$ must be chosen carefully (see Sec. IV), but we note here that the scale of $\mathbf{W}_{in}$ does not need to be tuned, as it is apparent from Eq. (3) that only the relative scale of $\mathbf{W}$ and $\mathbf{W}_{in}$ determines Λ_i.

The three parameters defined above—k, ρ, and σ—are the three hyperparameters that characterize the topology of the reservoir. We introduce a final parameter $\bar{\tau}$ in Sec. III A, which characterizes delays introduced along links between nodes. Together, these four hyperparameters describe the reservoirs that we investigate in this work.

A. Matching time scales with delays

The presence of the $-x_i$ term in Eq. (1) represents the sluggish response of the node, i.e., its inability to change its state instantaneously. This results in an effective propagation delay of a signal through the node. We can take advantage of this phenomenon by connecting chains of pairs of inverter gates between nodes. These inverter gates have dynamics described by Eqs. (1) and (2) and

$$\Lambda_i(X) = \begin{cases} 0 & \text{if } X = 1, \\ 1 & \text{if } X = 0. \end{cases} \tag{5}$$

Note that the propagation delay through these nodes depends both on γ_i and q_i, both of which are heterogeneous throughout the chip due to small manufacturing differences. We denote the mean propagation delay through the inverter gates by τ_{inv}, which we measure by recording the oscillation frequencies of variously sized loops of these gates. For the Arria 10 devices considered here,[28] we find $\tau_{inv} = 0.19 \pm 0.05$ ns.

We exploit the propagation delays by inserting chains of pairs of inverter gates in between reservoir nodes, thus creating a time-delayed network. We fix the mean delay $\bar{\tau}$ and randomly choose a delay time for each network link. This is similar to how the network topology is chosen by fixing certain hyperparameters and randomly choosing $\mathbf{W}$ and $\mathbf{W}_{in}$ subject to these parameters. The random delays are chosen from a uniform distribution between $\bar{\tau}/2$ and $3\bar{\tau}/2$ so that delays on the order of τ_{node} are avoided.

The addition of these delay chains is necessary because the time-scale of individual nodes is much faster than the speed at which synchronous FPGA logic can change the value of the input signal (see Sec. IV). Without any delays, it is impossible to match the time-scales of the input signal with the reservoir state, and we have poor RC performance. We find that the time-scales associated with the reservoir's fading memory are controlled by $\bar{\tau}$, as described in Sec. III B, thus demonstrating that we can tune the reservoir's time-scales with delay lines.

B. Fading memory

For the reservoir to learn about its input sequence, it is believed that it must possess the fading memory property (although more may be required for replicating long-term behavior[29]). Intuitively, this property implies that the reservoir state $\mathbf{X}(t)$ is a function of its input history, but is more strongly correlated with more recent inputs. More precisely, the fading memory property states that every reservoir state $\mathbf{X}(t_0)$ is uniquely determined by a left-infinite input sequence $\{\mathbf{u}(t) : t < t_0\}$.

The fading memory property is equivalent[17] to the statement that, for any two reservoir states $\mathbf{X}_1(t_0)$ and $\mathbf{X}_2(t_0)$ and input signal $\{\mathbf{u}(t) : t > t_0\}$, we have

$$\lim_{t\to\infty} ||\mathbf{X}_1(t) - \mathbf{X}_2(t)||_2 = 0. \tag{6}$$

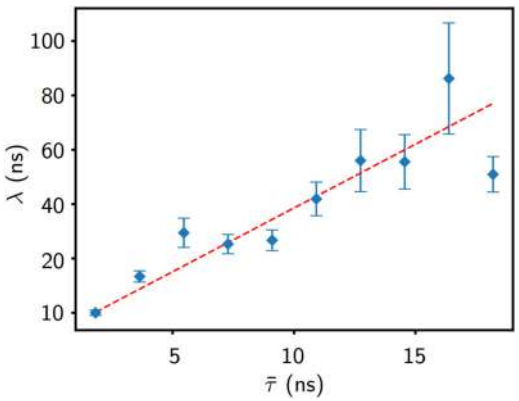

FIG. 2. Experimental observation of the fading memory property and decay time for varying $\bar{\tau}$. The network has 100 nodes and hyperparameters $k = 2$, $\rho = 1.5$, and $\sigma = 0.75$. Statistics are generated by testing five reservoirs for each set of hyperparameters. Vertical error bars represent the standard error of the mean. The relationship is approximately linear with a slope of 3.99 ± 0.45.

Also of interest is the characteristic time-scale over which this limit approaches zero, which may be understood as the Lyapunov exponent of the coupled reservoir-input system conditioned on the input.

We observe the fading memory property and measure the corresponding time-scale with the following procedure. We prepare two input sequences $\{\mathbf{u}_1(i\Delta t); -N \le i \le N\}$ and $\{\mathbf{u}_2(i\Delta t); -N \le i \le N\}$, where Δt is the input sample rate (see Sec. IV) and N is an integer such that $N\Delta t$ is sufficiently large. Each $\mathbf{u}_1(i\Delta t)$ is drawn from a random, uniform distribution between -1 and 1. For $i \ge 0$, $\mathbf{u}_2(i\Delta t) = \mathbf{u}_1(i\Delta t)$. For $i < 0$, $\mathbf{u}_2(i\Delta t)$ is drawn from a random, uniform distribution between -1 and 1. We drive the reservoir with the first input sequence and observe the reservoir response $\{\mathbf{X}_1(i\Delta t); -N \le i \le N\}$. After the reservoir is allowed to settle to its equilibrium state, we drive it with the second input sequence and observe $\{\mathbf{X}_2(i\Delta t); -N \le i \le N\}$. The reservoir is perturbed to effectively random reservoir states $\mathbf{X}_1(0)$ and $\mathbf{X}_2(0)$, because the input sequences are unequal for $i < 0$. For $i \ge 0$, the input sequences are equal, and the difference in Eq. (6) can be calculated.

For a given reservoir, this procedure is repeated 100 times with different input sequences. For each pair of sequences, the state difference is fit to $\exp(-t/\lambda)$, and the λ's are averaged over all 100 sequences. We call λ the reservoir's *decay time*. We find $\lambda > 0$ for every reservoir examined, demonstrating the usefulness of the chosen form of Λ_i in Eq. (3).

We explore the dependence of the decay time as a function of hyperparameter $\bar{\tau}$. As seen from Fig. 2, the relationship is approximately linear for fixed k, ρ, and σ. This is consistent with $\bar{\tau}$ being the dominate time-scale of the reservoir rather than τ_{node}, which is our motivation for including delay lines in our reservoir construction. The dependence of λ on the other hyperparameters defined in Sec. III is explored in Sec. VI along with corresponding results on a time-series prediction task.

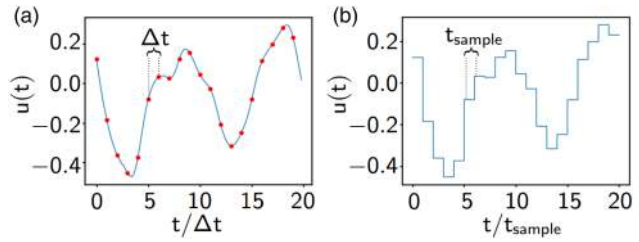

FIG. 3. A visualization of the discretization of $\mathbf{u}(t)$ necessary for hardware computation. (a) In general, the true input signal may be real-valued and defined over a continuous interval. (b) Due to finite precision and sampling time, the actual $\mathbf{u}(t)$ seen by the reservoir is held constant over intervals of duration t_{sample} and have finite vertical precision. For the prediction task, $v_d(t) = u(t)$, so the output must be discretized similarly.

IV. INPUT LAYER

As discussed in Sec. III, our reservoir implementation is an autonomous system without a global clock, allowing for continuously evolving dynamics. However, the input layer is a synchronous FPGA design that sets the state of the input signal $\mathbf{u}(t)$. Prior to operation, a sequence of values for $\mathbf{u}(t)$ is stored in the FPGA memory blocks. During the training period, the input layer sequentially changes the state of the input signal according to the stored values.

For the prediction task, the stored values of $\mathbf{u}(t)$ are observations of some time-series from $t = -T_{train}$ to $t = 0$. This signal maybe defined on the entire real interval $[-T_{train}, 0]$, but only a finite sampling may be stored in the FPGA memory and presented as the input to the reservoir. The signal may also take real values, but only a finite resolution at each sampling interval may be stored. The actual input signal $\mathbf{u}(t)$ in Fig. 1 is thus discretized in two ways:

- $\mathbf{u}(t)$ is held constant along intervals of length t_{sample};
- $\mathbf{u}(t)$ is approximated by an $n-$bit representation of real numbers.

A visualization of these discretizations is given in Fig. 3. Note that t_{sample} is a physical unit of time, whereas Δt has whatever units (if any) in which the non-discretized time-series is defined.

As pointed out in Sec. III, t_{sample} may be no smaller than the minimum time in which the clocked FPGA logic can change the state of the input signal, which is approximately 5 ns on the Arria 10 device considered here. However, we show in Sec. V that t_{sample} must be greater than or equal to τ_{out}, which generally cannot be made as short as 5 ns.

A. Binary representations of real data

The Boolean functions described by Eqs. (3) and (4) are defined according to Boolean values u_j, which are the bits in the $n-$bit representation of the input signal. If the elements of $\mathbf{W}_{in}$ are drawn randomly from a single distribution, then the reservoir state is as much affected by the least significant bit of $\mathbf{u}(t)$ as it is the most significant. This leads to the reservoir state being distracted by small differences in the input signal and fails to produce a working reservoir computer.

For a scalar input $u(t)$, we can correct this shortcoming by choosing the rows of $\mathbf{W}_{in}$ such that

$$\sum_j W_{in}^{i,j} u_j \approx \tilde{W}_{in}^i u, \tag{7}$$

where $\tilde{\mathbf{W}}_{in}$ is an effective input matrix with non-zero values drawn randomly between 1 and -1. The relationship is approximate in the sense that u is a real-number and u_j is a binary representation of that number. For the two complement representations, this is done by choosing

$$W_{in}^{i,j} = \begin{cases} -2^{(n-1)} \tilde{W}_{in}^i \text{ if } j = n, \\ +2^{(j-1)} \tilde{W}_{in}^i \text{ else.} \end{cases} \tag{8}$$

A disadvantage of the proposed scheme is that every bit in the representation of u must go to every node in the reservoir. If a node has k recurrent connections, then it must execute a $n + k$ to 1 Boolean function, as can be seen from Eq. (3). Boolean functions with more inputs take more FPGA resources to realize in hardware, and it takes more time for a compiler to simplify the function. We find that an 8-bit representation of u is sufficient for the prediction task considered here while maintaining achievable networks.

V. OUTPUT LAYER

Similar to the input layer, the output layer is constructed from synchronous FPGA logic. Its function is to observe the reservoir state and, based on a learned output matrix $\mathbf{W}_{out}$, produce the output $\mathbf{v}(t)$. As noted in Sec. II, this operation requires a time τ_{out} that we interpret as a propagation delay through the output layer and requires that $\mathbf{v}(t)$ be calculated from $\mathbf{X}(t - \tau_{out})$.

For the time-series prediction task, the desired reservoir output $\mathbf{v}_d(t)$ is just $\mathbf{u}(t)$. As discussed in Sec. IV, the input signal is discretized both in time and in precision so that the true state of the input signal is similar to the signal in Fig. 3(b). Thus, $\mathbf{v}(t)$ must be discretized in the same fashion. Note that, because the reservoir state $\mathbf{X}(t)$ is Boolean valued, a linear transformation $\mathbf{W}_{out}$ of the reservoir state is equivalent to a partial sum of the weights $\mathbf{W}_{out}$, where W_{out}^i is included in the sum only if $X_i(t) = 1$.

We find that the inclusion of a direct connection (see Sec. II and Fig. 1) greatly improves prediction performance. Though this involves a multiplication of 8-bit numbers, it only slightly increases τ_{out} because this multiplication can be done in parallel with the calculation of the addition of the Boolean reservoir state.

With the above considerations in mind, the output layer is constructed as follows: on the rising edge of a global clock with period t_{global}, the reservoir state is passed to a register in the output layer. The output layer calculates $\mathbf{W}_{out}\mathbf{X}$ with synchronous logic and in one clock cycle, where the weights $\mathbf{W}_{out}$ are stored in on-board memory blocks. The calculated output $\mathbf{v}(t)$ is passed to a register on the edge of the global clock. If $t > 0$, i.e., if the training period has ended, the input layer passes $\mathbf{v}(t)$ to the reservoir rather than the next stored value of $\mathbf{u}(t)$.

For $\mathbf{v}(t)$ to have the same discretized form as $\mathbf{u}(t)$, we must have the global clock period t_{global} be equal to the input

period t_{sample}, which means the fastest our reservoir computer can produce predictions is once every $\max\{\tau_{out}, t_{sample}\}$. While t_{sample} is independent of the size of the reservoir and precision of the input, τ_{out} in general depends on both. We find that $\tau_{out} = 6.25$ ns is the limiting period for a reservoir of 100 nodes, an 8-bit input precision, and the Arria 10 FPGA considered here. Our reservoir computer is therefore able to make predictions at a rate of 160 MHz, which is currently the fastest prediction rate of any real-time RC to the best of our knowledge.

VI. REAL-TIME PREDICTION

We apply the complete reservoir computer—the autonomous reservoir and synchronous input and output layers—to the task of predicting a chaotic time-series. To quantify the performance of our prediction algorithm, we compute the normalized root-mean-square error (NRMSE) over one Lyapunov time T, where T is the inverse of the largest Lyapunov exponent. The $NRMSE_T$ is therefore defined as

$$NRMSE_T = \sqrt{\frac{\sum_{t=0}^{T}[u(t) - v(t)]^2}{T\sigma^2}}, \tag{9}$$

where σ^2 is the variance of $u(t)$.

To train the reservoir computer, the reservoir is initially driven with the stored values of $u(t)$ as described in Sec. III and the reservoir response is recorded. This reservoir response is then transferred to a host PC. The output weights $\mathbf{W}_{out}$ are chosen to minimize

$$\sum_{t=-T_{train}}^{0} [u(t) - v(t)]^2 + r|\mathbf{W}_{out}|^2, \tag{10}$$

where r is the ridge regression parameter and is included in Eq. (6) to discourage over-fitting to the training set. The value of r is chosen by leave-one-out cross validation on the training set. We choose a value of T_{train} so that 1500 values of $u(t)$ are used for training.

A. Generation of the Mackey-Glass system

The Mackey-Glass system is described by the time-delay differential equation

$$\dot{u}(t) = \beta \frac{u(t-\tau)}{1 + u^n(t-\tau)} - \gamma u(t), \tag{11}$$

where β, γ, τ, and n are positive, real constants. The Mackey-Glass system exhibits a range of ordered and chaotic behavior. A commonly chosen set of parameters is $\beta = 0.2$, $\gamma = 0.1, \tau = 17, n = 10$ for which Eq. (7) exhibits chaotic behavior with an estimated largest Lyapunov exponent of 0.0086 ($T = 116$).

Equation (10) is integrated using a 4^{th}-order Runge-Kutta method, and the resulting series is normalized by shifting by -1 and passing $u(t)$ through a hyperbolic tangent function as in Ref. 11, resulting in a variance $\sigma^2 = 0.046$. As noted in Sec. III, $u(t)$ must be discretized according to Fig. 3(b). We find an optimal temporal sampling of $\Delta t = 5$ as in Fig. 3(a).

VII. RESULTS ANALYSIS

The reservoirs considered here are constructed from random connection matrices $\mathbf{W}$ and $\mathbf{W}_{in}$. However, we seek to understand the reservoir properties as functions of the hyperparameters that control the distributions of these random matrices. Recall from Sec. III that these hyperparameters are:

- the largest absolute eigenvalue of $\mathbf{W}$, denoted by ρ;
- the fixed in-degree of each node, denoted by k;
- the mean delay between nodes, denoted by $\bar{\tau}$;
- and the number of nodes which receive the input signal, denoted by σ.

Because t_{sample} and, consequently, the global temporal properties of the predicting reservoir are coupled to the network size N, we fix $N = 100$ and consider the effects of varying the four hyperparameters given above.

Obviously, many instances of $\mathbf{W}_{in}$ and $\mathbf{W}$ have the same hyperparameters. We therefore consider the dynamical properties considered in this section as well as prediction performance to be random variables whose mean and variance we wish to investigate. For each set of reservoir parameters,

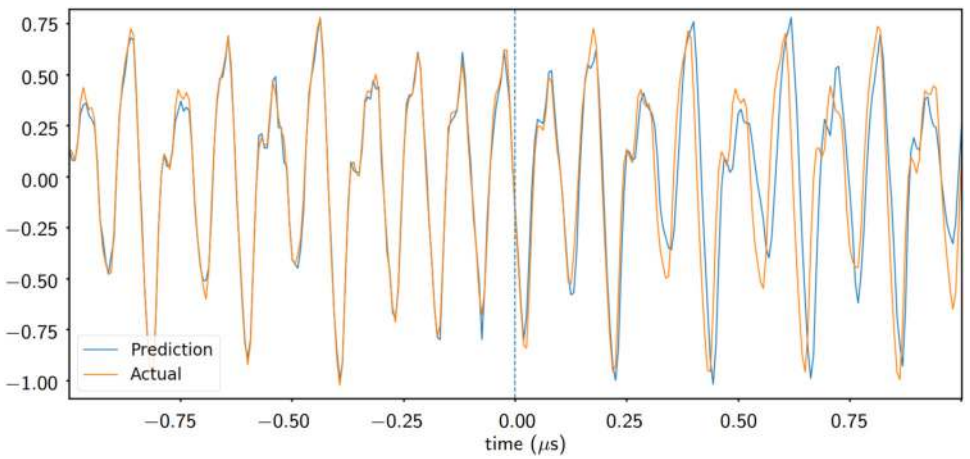

FIG. 4. An example of the output of a trained reservoir computer. Autonomous generation starts at $t = 0$. The target signal is the state of the Mackey-Glass system described by Eq. (11). The particular hyperparameters are $(\rho, k, \bar{\tau}, \sigma) = (1.5, 2, 11\text{ ns}, 0.5)$.

5 different reservoirs are created and each tested 5 times at the prediction task.

For optimal choice of reservoir parameters $(\rho, k, \bar{\tau}, \sigma) = (1.5, 2, 11\text{ ns}, 0.5)$, we measure $NRMSE = 0.028 \pm 0.010$ over one Lyapunov time. The predicted and actual signal trajectories for this reservoir are in Fig. 4. For comparison to other works, we prepared ESN as in Ref. 11 toward with the same network size (100 nodes) and training length (1500 samples) and find a $NRMSE_T = 0.057 \pm 0.007$.

A. Spectral radius

The spectral radius ρ controls the scale of the weights $\mathbf{W}$. Though there are many ways to control this scale (such as tuning the bounds of the uniform distribution[30]), ρ is often seen to be a useful way to characterize a classical ESN.[31,32] Optimizing this parameter has been critical in many applications of RC, with a spectral radius near 1 being a common starting point. More abstractly, the memory capacity has been demonstrated to be maximized at $\rho = 1.0$ from numerical experiments[33] and it has been shown that ESNs do not have the fading memory property for all inputs for $\rho > 1.0$.[17]

It is not immediately clear that ρ will be a similarly useful characterization of our Boolean networks, since the activation function [see Eq. (1)] is discontinuous and includes time-delays—both factors which are typically not assumed to be true in the current literature. Nonetheless, we proceed with this scaling scheme and investigate the decay times and prediction performance properties of our reservoirs as we vary this parameter.

We see from Fig. 5 that the performance on the Mackey-Glass prediction task is indeed optimized at $\rho = 1.0$. However, performance is remarkably flat, quite unlike more traditional ESNs. The performance will obviously fail as $\rho \to 0$ (corresponding to no recurrent connections) and as $\rho \to \infty$ (corresponding to no input connections), and it appears that a range of ρ in between yield similar performance.

This flatness in prediction performance is reflected in measures of the dynamics of the reservoir as seen in Figs. 5(a) and 5(b). Note that the decay time of the reservoir decreases for smaller ρ. This behavior is expected, because, as the network becomes more loosely self-coupled, it is effectively more strongly coupled to the input signal, and thus will more quickly forget previous inputs. More surprising is the flatness beyond $\rho = 1.0$, which mirrors flatness in the performance error in this region of spectral radii.

We propose that this insensitivity to ρ is due to the nature of the activation function in Eq. (3). Note that, because of the flat regions of the Heaviside step function and the fact that the Boolean state variables take discrete values, there exists a range of weights that correspond to precisely the same Λ_i for a given node. Thus, the network dynamics are less sensitive to the exact tuning of the recurrent weights than in an ESN.

B. Connectivity

The second component to characterizing $\mathbf{W}$ is the in-degree k of the nodes, which is the density of non-zero entries in the row vectors of $\mathbf{W}$. Because the Λ_i's are populated by explicit calculation of the functions in Eq. (3) and because

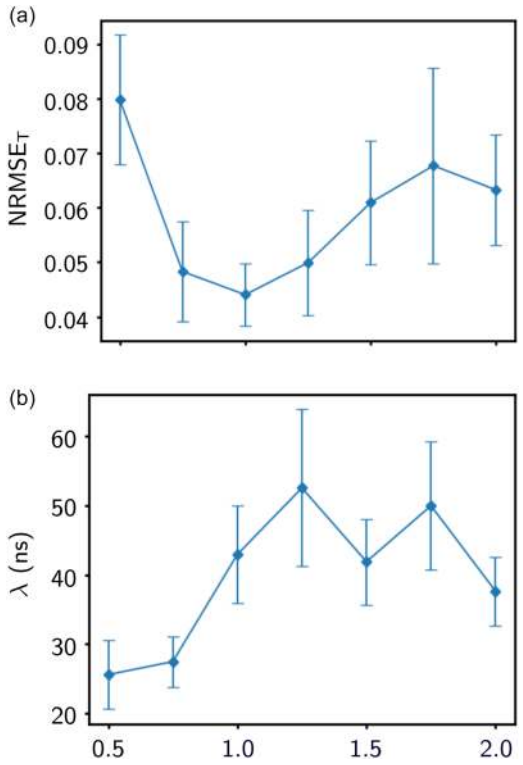

FIG. 5. Prediction performance and fading memory of reservoirs with $(k, \bar{\tau}, \sigma) = (2,\ 11\text{ ns},\ 0.75)$ and varying ρ. (a) Somewhat consistent with observations in echo-state networks, ρ near 1.0 appears to be a good choice. However, a much wider range of ρ suffice as well. (b) As ρ becomes small and the reservoir becomes more strongly coupled to the input, the reservoir more quickly forgets previous inputs. The decay time levels out above $\rho = 1.0$. Note that λ is everywhere the same order of magnitude as $\bar{\tau}$.

larger Λ_i's require more resources to realize in hardware, it is advantageous to limit k. We therefore ensure that each node has fixed k rather than simply some mean degree that is allowed to vary.

From the study of purely Boolean networks with discrete-time dynamics (i.e., dynamics defined by a map rather than a differential equation), a transition from order to chaos is seen in a number of network motifs at $k = 2$.[34,35] In fact, Hopefield type nodes are seen to have this critical connectivity in the explicit context of RC.[30] The connectivity is a commonly optimized hyperparameter in the context of ESNs as well[17,36] with the common heuristic that low-connectivity (1%–5% of N) promotes a richer reservoir response.

From the above considerations, we study the reservoir dynamics and prediction performance as we vary k =1–4. From Fig. 6, we see stark contrasts from the picture of RC with a Boolean network in discrete time. First, the reservoirs remain in the ordered phase for k =2–4, which clearly demonstrates that the real-valued nature of the underlying dynamical variables in Eq. (3) is critically important to the network dynamics.

We see further in Fig. 6(b) that the mean decay time increases with increasing k, i.e., the network takes longer time to forget past inputs when the nodes are more densely connected. This phenomenon is perhaps understood by the increased number of paths in networks with higher k. These paths provide more avenues for information about previous

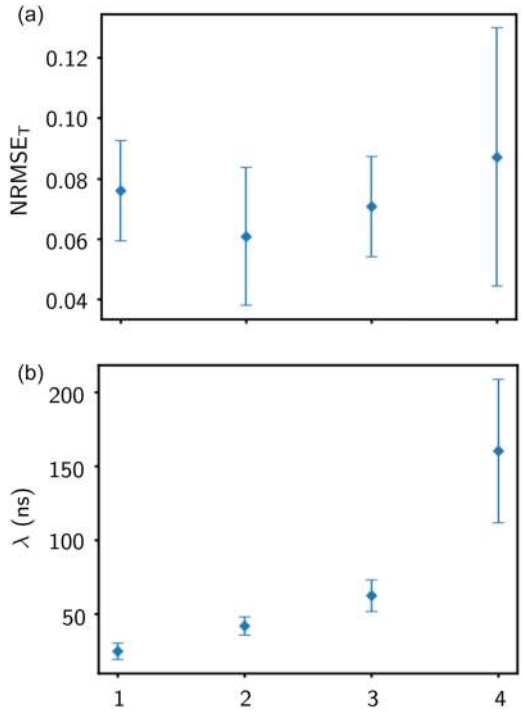

FIG. 6. Prediction performance and fading memory of reservoirs with $(\rho, \bar{\tau}, \sigma) = (1.5, 11\text{ ns}, 0.75)$ and varying k. (a) We see effectively no difference over this range, contrary to intuitions from studies of Boolean networks in discrete time. (b) For $k = 1$, λ is approximately equal to $\bar{\tau}$. However, as we increase k to 4, both the mean and variance of λ approaches almost an order of magnitude larger than $\bar{\tau}$.

network states to propagate, thus prolonging the decay of the difference in Eq. (6). The variance in decay time also significantly increases for increasing k. This may be an indicator of eventual criticality for large enough k.

Given the strong differences in reservoir dynamics between $k = 1, 4$, it is surprising that no significant difference at the prediction task is detected. However, it is useful for the design of efficient reservoirs to observe that very sparsely connected reservoirs suffice for complicated tasks. As noted in Sec. IV, nodes with more inputs require more resources to realize in hardware and more processing time to compute the corresponding Λ_i in Eq. (3).

C. Mean delay

As argued in Sec. III, adding time-delays along the network links increases the characteristic time scale of the network. We distribute delays by randomly choosing, for each network link, a delay time from a uniform distribution from $\bar{\tau}/2$ to $3\bar{\tau}/2$. The shape of this distribution is chosen to fix the mean delay time while keeping the minimum delay time above the characteristic time of the nodes themselves.

In Fig. 7, we compare the prediction performance vs. $\bar{\tau}$. Note that this parameter is most critical in achieving good prediction performance in the sense that $\bar{\tau}$ being comparable to τ_{node} yields poor performance. However, the performance is flat past a certain minimum $\bar{\tau}$ near 8.5 ns. This point is important to identify, as adding more delay elements than necessary increases the number of FPGA resources needed to realize the network.

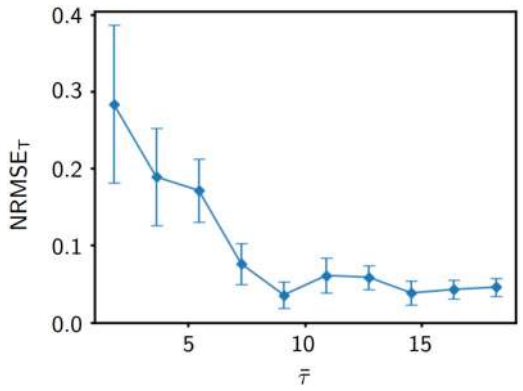

FIG. 7. Prediction performance of reservoirs with $(\rho, k, \sigma) = (1.5, 2, 0.75)$ and varying $\bar{\tau}$. The NRMSE decreases until approximately $\bar{\tau} = 9.5$, after which point it remains approximately constant.

D. Input density

We finally consider the effect of tuning the proportion of reservoir nodes that are connected to the input signal. This proportion is often assumed to be 1,[36] although recent studies have shown a smaller fraction to be useful in certain situations, such as predicting the Lorenz system.[37]

We observe from Fig. 8(a) that an input density of 0.5 performs better than input densities of 0.25, 0.75, and 1.0. We note from Fig. 8(b) that this corresponds to the point of longest decay time. The decreasing decay time with higher

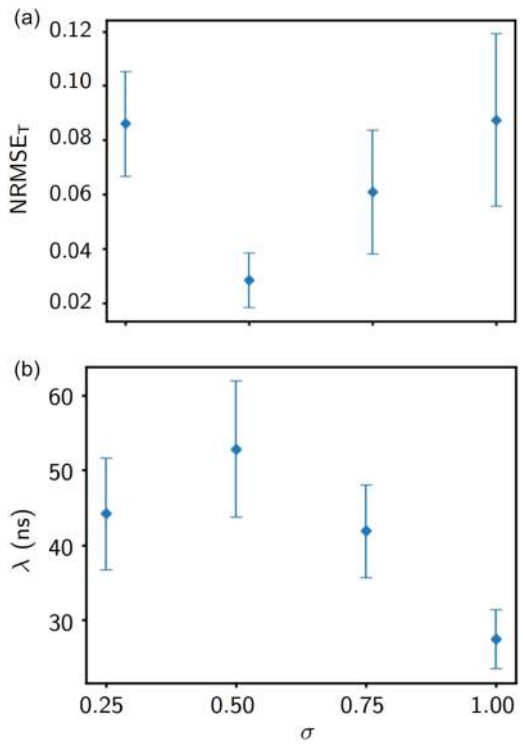

FIG. 8. Prediction performance and fading memory of reservoirs with $(\rho, k, \bar{\tau}) = (1.5, 2, 11\text{ ns}, 0.75)$ and varying ρ. (a) Choosing $\sigma = 0.5$ improves prediction performance by a factor of 3 over the usual choice of $\sigma = 1.0$ (b) With larger σ, the reservoir is more strongly coupled to the input signal. Consequently, λ decreases, signifying that the reservoir is more quickly forgetting previous inputs.

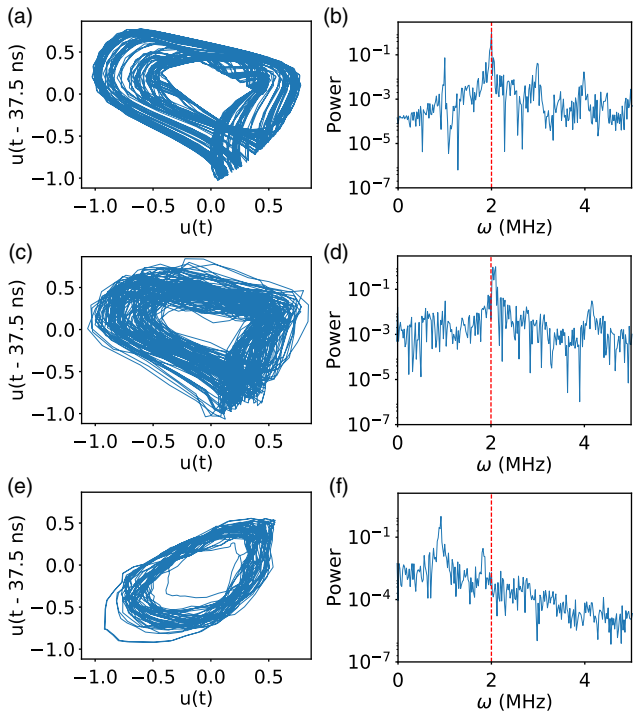

FIG. 9. Phase-space representations and power spectra of the attractors of Eq. (11) and trained reservoirs. (a) The true attractor and (b) normalized power spectrum of the Mackey-Glass system, as presented to the reservoir. (c) The attractor and (d) normalized power spectrum for a reservoir whose long-term behavior is similar to the true Mackey-Glass system. Although "fuzzy," the attractor remains near the true attractor. The power spectrum shows a peak 0.10 MHz away from the true peak. The hyperparameters for this reservoir are $(\rho, k, \bar{\tau}, \sigma) = (1.5, 2, 11\,\text{ns}, 0.75)$. (e) The attractor and (f) normalized power spectrum of a reservoir whose long-term behavior is different than the true Mackey-Glass system. The dominate frequency of the true system is highly suppressed, while a lower-frequency mode is amplified. The hyperparameters for this reservoir are $(\rho, k, \bar{\tau}, \sigma) = (1.5, 4, 11\,\text{ns}, 0.75)$. The dashed, red line in the power spectrum plots indicates the peak of the spectrum in the true Mackey-Glass system.

input densities 0.75 and 1.0 are consistent with the expectation that reservoirs that are more highly coupled to the input signal will forget previous inputs more quickly.

It is apparent from Fig. 8(b) that the input density is a useful characterization of the RC scheme, impacting the fading memory properties of the reservoir-input system and ultimately improving performance by a factor of 3 when compared to a fully dense input matrix. This result suggests the input density to be a hyperparameter deserving more attention in general contexts.

E. Attractor reconstruction

Prediction algorithms are commonly evaluated on their short-term prediction abilities as we have done so far in this section. The predicted and actual signal trajectories will always diverge in the presence of chaos due to the positivity of at least one Lyapunov exponent. However, it has been seen recently that reservoir computers[19] and other neural network prediction schemes[38] can have similar long-term behavior as the target system. In particular for ESNs, it has been seen that different reservoirs can have similar short-term prediction capabilities, but very different long-term behavior, with some reservoirs capturing the climate of the Lorenz system and others eventually collapsing onto a non-chaotic attractor.[19] This phenomenon has recently been explained in terms of generalized synchronization—a stronger condition than fading memory.[29]

To observe a similar phenomenon in the RC scheme considered here, we allow a trained reservoir to evolve for 100 Lyapunov times (about 15 μs) beyond the training period. The last half of this period is visualized in time-delay phase-space to see if the climate of the true Mackey-Glass system is replicated.

Our results show phenomena consistent with previous observations in ESNs. Figure 9(a) shows the true attractor of Eq. (11), which has fractal dimension and is non-periodic. Figure 9(b) shows the attractor of a well-chosen autonomous, Boolean reservoir. Although the attractor is "fuzzy," the trajectory remains on a Mackey-Glass-like shape well beyond the training period. On the other hand, a reservoir with similar short-term prediction error is shown in Fig. 9(c). Although this network is able to replicate the short-term dynamics of Eq. (11), its attractor is very unlike the true attractor in Fig. 9(a). This results shows that, even in the presence of noise inherent in physical systems, the autonomous Boolean reservoir can learn the long-term behaviors of a complicated, chaotic system.

VIII. DISCUSSION AND CONCLUSION

We conclude that an autonomous, time-delay, Boolean network serves as a suitable reservoir for RC. We have demonstrated that such a network can perform the complicated task of predicting the evolution of a chaotic dynamical system with comparable accuracy to software-based RC. We have demonstrated the state-of-the-art speed with which our reservoir computer can perform this calculation, exceeding previous hardware-based solutions to the prediction problem. We have demonstrated that, even after the trained reservoir computer deviates from the target trajectory, the attractor stays close to the true attractor of the target system.

This work demonstrates that fast, real-time computation with autonomous dynamical systems is possible with readily-available electronic devices. This technique may find applications in the design of systems that require estimation of the future state of a system that evolves on a nanosecond to microsecond time scale, such as the evolution of cracks through crystalline structures or the motion of molecular proteins.

ACKNOWLEDGMENTS

We gratefully acknowledge discussions of this work with Roger Brockett, Michele Girvan, Brian Hunt, and Edward Ott and the financial support of the U.S. Army Research Office Grant No. W911NF-12-1-0099.

APPENDIX A: REALIZING THE RESERVOIR ON AN FPGA

In this Appendix, we present the hardware description code for the reservoir nodes, delay lines, and a small reservoir.

The code is written in *Verilog* and compiled using Altera's *Quartus Prime* software. Some parts of the code depend on the number of reservoir nodes N, the node in-degree k, and the number of bits n used to represent the input signal $u(t)$. We give explicitly the code only for $N = 3$, $k = 2$, and $n = 1$, but generalizations are straightforward.

As discussed in Sec III, reservoir nodes implement a Boolean function $\Lambda_i : \mathbf{Z}_2^{k+n} \to \mathbf{Z}_2$ of the form given in Eq. (3). Each Boolean function can be defined by a Boolean string of length 2^{k+n} that specifies the look-up-table (LUT) corresponding of the Boolean function. For example, the AND function maps $\mathbf{Z}_2^2 \to \mathbf{Z}_2$ and has the LUT defined in Fig. 10. The Boolean string that defines the AND function is 0001 as can be seen from the right-most column of the LUT.

The code given in Fig. 11 generates a node with Boolean function based on any LUT of length $2^3 = 8$. The module **node** is declared in line 1 with inputs **node_in** and output **node_out**. The width of **node_in** is 3 bits as specified in line 3. The parameter **lut** is declared in line 2. Note that it is initialized to some value as required by *Quartus*, but this value is changed whenever a node is declared within the larger code that defines the complete reservoir.

The main part of the code is within an **always @(*)** block, which creates an inferred sensitivity list and is used to create arbitrary combinational logic. Line 7 specifies that values before the colon in the proceeding lines correspond to **node_in**. The statement following the colon determines which value is assigned to **node_out**. In effect, line 8 simply specifies that, whenever the value of **node_in** is a 3-bit string equal to 000, the value of **node_out** is whatever the value of **lut[7]** is. For example, if we create an instance of the module **node** with parameter **lut=8'b00000001**, then the node will execute the 3 input AND function.

As discussed in Sec. IV, delay lines are created as chains of pairs of inverter gates. Such a chain of length $2m$ is created with the code in Fig. 12. Similarly to the **node** module, the **delay_line** module is declared in line 1 with the input **delay_in** and output **delay_out**. It has a parameter m which specifies the number of pairs in the chain and can be changed

```
1  module node(node_in, node_out);
2  parameter lut = 8'b00000001;
3  input[2:0] node_in;
4  output reg node_out;
5
6  always @(*) begin
7    case (node_in)
8      3'b000 : node_out = lut[7];
9      3'b001 : node_out = lut[6];
10     3'b010 : node_out = lut[5];
11     3'b011 : node_out = lut[4];
12     3'b100 : node_out = lut[3];
13     3'b101 : node_out = lut[2];
14     3'b110 : node_out = lut[1];
15     3'b111 : node_out = lut[0];
16   endcase
17 end
18 endmodule
```

FIG. 11. Verilog code for a generic node that can implement any 3-input Boolean function, specified by a Boolean string of length 8.

when calling a specific instance of **delay_line**. A number of wires are declared in line 5 and will be used as the inverter gates. Note the important directive **/*synthesis keep*/**, which instructs the compiler to not simplify the module by eliminating the inverter gates. This is necessary, because otherwise the compiler would realize that **delay_line**'s function is trivial and remove all of the inverter gates.

Lines 7–8 specify the beginning and end of the delay chain as the **delay_in** and **delay_out**, respectively. Lines 10–16 use a **generate** block to create a loop that places inverter gates in between **delay_in** and **delay_out**, resulting in a delay chain of length $2m$.

The **reservoir** module is the code that creates N instances of **node** and connects them Nk instances of **delay_line**. As an illustrative example, consider a 3-node reservoir with the

x	y	x AND y
0	0	0
0	1	0
1	0	0
1	1	1

FIG. 10. The LUT for the AND function. It can be specified by the Boolean string that makes up the right-most column.

```
1  module delay_line(delay_in, delay_out);
2  parameter m = 15;
3  input delay_in;
4  output delay_out;
5  wire [2*m-1:0] delay /*synthesis keep */;
6
7  assign delay[0] = ~delay_in;
8  assign delay_out = delay[2*m-1];
9
10 genvar i;
11 generate
12   for (i=0;i<2*m-1;i=i+1)
13   begin : generate_delay
14     assign delay [i+1] = ~delay[i];
15   end
16 endgenerate
17 endmodule
```

FIG. 12. Verilog code for a delay line with $2m$ inverter gates.

```
1 module reservoir (u, x);
2 parameter N = 3;
3 parameter k = 2;
4 parameter m = 1;
5 input [m-1:0] u;
6 output [N-1:0] x;
7 wire [N*k-1:0] x_tau;
8
9 node #(8'b01111111) node_0 ({u, x_tau[0], x_tau
      [1]}, x[0]);
10 node #(8'b01000000) node_1 ({u, x_tau[2], x_tau
      [3]}, x[1]);
11 node #(8'b01001101) node_2 ({u, x_tau[4], x_tau
      [5]}, x[2]);
12
13 delay_line #(10) delay_0 (x[0], x_tau[0]);
14 delay_line #(15) delay_1 (x[1], x_tau[1]);
15 delay_line #(6) delay_2 (x[0], x_tau[2]);
16 delay_line #(7) delay_3 (x[2], x_tau[3]);
17 delay_line #(10) delay_4 (x[0], x_tau[4]);
18 delay_line #(12) delay_5 (x[1], x_tau[5]);
19 endmodule
```

FIG. 13. Verilog code describing a simple reservoir. The connections and LUTs are determined from Eqs. (3) and (A1)–(A3). Lines 9–11 declare 3 nodes. Lines 13–18 declare delay lines that connect them.

following parameters

$$\mathbf{W} = \begin{bmatrix} 0.1 & 0.3 & 0 \\ -0.2 & 0 & 0.1 \\ -0.3 & 0.2 & 0 \end{bmatrix}, \tag{A1}$$

$$\mathbf{W}_{in} = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.2 \end{bmatrix}, \tag{A2}$$

$$\boldsymbol{\tau} = \begin{bmatrix} 10 & 15 & 0 \\ 6 & 0 & 7 \\ 12 & 10 & 0 \end{bmatrix} \tag{A3}$$

and only a 1-bit representation of $u(t)$. When we pass $u(t)$ and $x(t)$ into the **node** module, we index such that $u(t)$ comes first, as seen from the **reservoir** module below.

With Eqs. (3) and (A1)–(A3), the LUTs for each node can be explicitly calculated as 01111111, 0100000000, and 01001101 for nodes 1–3, respectively. The matrix $\boldsymbol{\tau}$ specifies the delays in integer multiples of $2\tau_{inv}$. A network with this specification is realized by the module **reservoir** in Fig. 13 and the **node** and **delay_in** modules described in this section.

Like the other modules, **reservoir** requires a module declaration, parameter declarations, and input/output declarations. Here, we also declare a wire **x_tau** that is the delayed reservoir state. In lines 9–11, the nodes are declared with the appropriate parameters and connections and are named **node_0**, **node_1**, and **node_2** respectively. The 6 delay lines are declared and named in lines 13-18.

APPENDIX B: SYNCHRONOUS COMPONENTS

In this Appendix, we discuss the details of the synchronous components that interact with the autonomous reservoir. These components regulate the reservoir input signal, the operation mode (training or autonomous), the calculation of the output signal, and record the reservoir state.

```
1 module reservoir_computer(clk);
2 input clk;
3
4 wire mode;
5 wire [m-1:0] u;
6 wire [2*m*(N+1)-1:0] W_out;
7 wire [N-1:0] x;
8 wire [m-1:0] v;
9
10 reg [N-1:0] x_reg;
11 reg [m-1:0] v_reg;
12 wire [m-1:0] u_v;
13
14 sampler sampler_1 (clk, mode, u, W_out);
15 player player_1 (clk, x, v);
16
17 assign u_v = mode ? u : v;
18
19 reservoir reservoir_1 (u_v, x);
20
21 output_layer output_layer 1 (W_out, u_v, x_reg,
      v);
22
23 always @(posedge clk) begin
24   x_reg <= x;
25   v_reg <= v;
26 end
27 endmodule
```

FIG. 14. Verilog code describing the reservoir computer. It contains the **reservoir** module discussed in App. A and various synchronous components.

Crucial to successful operation is access to a **sampler** module that reads data from the reservoir and a **player** module that writes data into the reservoir. The details of these modules are not discussed here as they depend on the device and the application of the reservoir computer. We assume that these modules are synchronized by a global clock **clk** such that **sampler** (**player**) reads (writes) data on the rising edge of **clk**,

In Fig. 14, we present a sample Verilog code for a high-level module **reservoir_computer** containing the reservoir and synchronous components. An instance of a **sampler** module is coupled to a global clock **clk** and outputs an m-bit wide signal **u**, a 1 bit signal **mode** that determines the mode of operation for the reservoir, and a $2m(N+1)$-bit wide signal **W_out** that determines the output weight matrix. An instance of a **player** module is also coupled to a global clock **clk** and inputs an N-bit wide signal **x** and a m-bit wide signal v. Depending on how these modules are implemented, they may also be coupled to other components, such as on-board memory or other FPGA clocks.

As seen in line 17, the state of **mode** determines whether **u** or **v** drives the reservoir. This bit is set to 1 during training and 0 after training to allow the reservoir to evolve autonomously (see Fig. 1).

clk registers **x** and **v** so that **output_layer** sees a value of **x** that is constant throughout one period t_{sample} and outputs a value **v** that is constant over that same interval (see Fig. 3). The module **output_layer** performs the operation $\mathbf{W}_{out}(\mathbf{x}, \mathbf{u})$, as described in Sec. V. **W_out** is a flattened array of the $N+1$ output weights represented by $2m$ bits, with the extra bits being necessary to avoid errors in the intermediate addition calculations.

[1]H. Zhang, Z. Wang, and D. Liu, IEEE Trans. Neural Netw. Learn. **25**, 1229 (2014).
[2]M. Lukoševičius and H. Jaeger, Comput. Sci. Rev. **3**, 127 (2009).
[3]D. Gauthier, "Reservoir computing: Harnessing a universal dynamical system," Siam News **51**(2), 12 (2018).
[4]K.-i. Funahashi and Y. Nakamura, Neural. Netw. **6**, 801 (1993).
[5]K. Hornik, M. Stinchcombe, and H. White, Neural Netw. **2**, 359 (1989).
[6]S. Hochreiter, Y. Bengio, and P. Frasconi, in *Field Guide to Dynamical Recurrent Networks*, edited by J. Kolen and S. Kremer (IEEE Press, 2001).
[7]D. Li, M. Han, and J. Wang, IEEE Trans. Neural Netw. Learn. **23**, 787 (2012).
[8]H. Jaeger, in *Advances in Neural Information Processing Systems* (MIT Press, 2003), pp. 609–616.
[9]C. Fernando and S. Sojakka, in *European Conference on Artificial Life* (Springer, Berlin, 2003), pp. 588–597.
[10]L. Larger, M. C. Soriano, D. Brunner, L. Appeltant, J. M. Gutiérrez, L. Pesquera, C. R. Mirasso, and I. Fischer, Opt. Express **20**, 3241 (2012).
[11]K. Vandoorne, W. Dierckx, B. Schrauwen, D. Verstraeten, R. Baets, P. Bienstman, and J. Van Campenhout, Opt. Express **16**, 11182 (2008).
[12]J. Nakayama, K. Kanno, and A. Uchida, Opt. Express **24**, 8679 (2016).
[13]L. Larger, A. Baylón-Fuentes, R. Martinenghi, V. S. Udaltsov, Y. K. Chembo, and M. Jacquot, Phys. Rev. X **7**, 011015 (2017).
[14]J. Bueno, D. Brunner, M. C. Soriano, and I. Fischer, Opt. Express **25**, 2401 (2017).
[15]F. Duport, B. Schneider, A. Smerieri, M. Haelterman, and S. Massar, Opt. Express **20**, 22783 (2012).
[16]P. Antonik, M. Hermans, F. Duport, M. Haelterman, and S. Massar, Proc. SPIE **9732**, 97320B (2016).
[17]H. Jaeger, GMD: German National Research Center for Information Technology Technical Report No. 148, Bonn, Germany, 2001, p. 13.
[18]W. Maass, T. Natschläger, and H. Markram, Neural Comput. **14**, 2531 (2002).
[19]J. Pathak, Z. Lu, B. R. Hunt, M. Girvan, and E. Ott, Chaos **27**, 121102 (2017).
[20]B. Schrauwen, J. Defour, D. Verstraeten, and J. Van Campenhout, in *International Conference on Artificial Neural Networks* (Springer, Berlin, 2007), pp. 471–479.
[21]F. Wyffels and B. Schrauwen, Neurocomputing **73**, 1958 (2010).
[22]L. Appeltant, M. C. Soriano, G. Van der Sande, J. Danckaert, S. Massar, J. Dambre, B. Schrauwen, C. R. Mirasso, and I. Fischer, Nat. Commun. **2**, 468 (2011).
[23]B. Schrauwen, M. D'Haene, D. Verstraeten, and J. Van Campenhout, Neural Netw. **21**, 511 (2008).
[24]M. L. Alomar, V. Canals, N. Perez-Mora, V. Martínez-Moll, and J. L. Rosselló, Comput. Intell. Neurosci. **2016**, 15 (2016).
[25]R. Zhang, H. L. d. S. Cavalcante, Z. Gao, D. J. Gauthier, J. E. Socolar, M. M. Adams, and D. P. Lathrop, Phys. Rev. E **80**, 045202 (2009).
[26]N. D. Haynes, M. C. Soriano, D. P. Rosin, I. Fischer, and D. J. Gauthier, Phys. Rev. E **91**, 020801 (2015).
[27]L. Glass and S. A. Kauffman, J. Theor. Biol. **39**, 103 (1973).
[28]We use an Arria 10 SX 10AS066H3F34I2SG chip for the results discussed in this paper.
[29]Z. Lu, B. R. Hunt, and E. Ott, Chaos **28**, 061104 (2018).
[30]L. Büsing, B. Schrauwen, and R. Legenstein, Neural Comput. **22**, 1272 (2010).
[31]K. Caluwaerts, F. Wyffels, S. Dieleman, and B. Schrauwen, in *The 2013 International Joint Conference on Neural Networks (IJCNN)* (IEEE, 2013), pp. 1–6.
[32]M. Lukoševičius, in *Neural Networks: Tricks of the Trade* (Springer, Berlin, 2012), pp. 659–686.
[33]D. Verstraeten, B. Schrauwen, M. d'Haene, and D. Stroobandt, Neural Netw. **20**, 391 (2007).
[34]B. Derrida and Y. Pomeau, Europhys. Lett. **1**, 45 (1986).
[35]T. Rohlf and S. Bornholdt, Physica A **310**, 245 (2002).
[36]H. Jaeger, *Tutorial on Training Recurrent Neural Networks, Covering BPPT, RTRL, EKF and the "Echo State Network" Approach* (GMD-Forschungszentrum Informationstechnik, Bonn, 2002), Vol. 5.
[37]J. Pathak, A. Wikner, R. Fussell, S. Chandra, B. R. Hunt, M. Girvan, and E. Ott, Chaos **28**, 041101 (2018).
[38]J. Qiao, G. Wang, W. Li, and X. Li, Neural Netw. **104**, 68 (2018).